Chapter 09: Dynamic Memory Allocation

ENG3091

Khiem Nguyen

Contents

1 Why dynamic memory? 1
- 1.1 Three ways to get memory . . . 1
- 1.2 When automatic memory isn't enough . . . 2
- 1.3 Overflowing the stack . . . 2
- 1.4 The heap . . . 3

2 `new` and `delete` 3
- 2.1 Allocating one object with `new` . . . 3
- 2.2 Giving memory back with `delete` . . . 3
- 2.3 `delete` and `nullptr` . . . 4
- 2.4 Dangling pointers, again . . . 5
- 2.5 Two pointers, one object . . . 5

3 Memory leaks 6
- 3.1 A memory leak . . . 6
- 3.2 Why leaks matter . . . 6
- 3.3 Catching leaks with AddressSanitizer . . . 6
- 3.4 Three ways to leak . . . 7

4 Dynamic arrays 8
- 4.1 Allocating an array with `new[]` . . . 8
- 4.2 What `new int[length]` gives you . . . 8
- 4.3 `new[]` needs `delete[]` . . . 9
- 4.4 `new` can fail . . . 9

5 Let someone else `delete` 10
- 5.1 `std::vector`: a dynamic array that cleans up after itself . . . 10
- 5.2 Smart pointers: `std::unique_ptr` . . . 10
- 5.3 So why learn `new` and `delete` at all? . . . 11

6 Wrapping up 11
- 6.1 Summary . . . 11
- 6.2 Keep learning . . . 11

1 Why dynamic memory?

1.1 Three ways to get memory

C++ gives memory to objects in three ways. You've already met two of them:

Allocation	For	Memory is obtained…	…and given back
Static	global and `static` variables	once, when the program starts	when the program ends
Automatic	local variables, parameters	when the block is entered	automatically, at its `}`
Dynamic	whatever you ask for	when *you* ask, with `new`	when *you* say, with `delete`

Static and automatic allocation have two things in common:

- the **size** of every object must be known when the program is **compiled**;
- memory is obtained and given back **automatically**.

Most of the time that's exactly what we want. But not always.

1.2 When automatic memory isn't enough

Problem 1: we don't know the size in advance. How many measurements are in the data file? How long is the user's name? How many particles are in the simulation right now? We find out only when the program **runs**.

Guessing a maximum, "500 records should be enough", wastes memory when there are fewer, and breaks when there are more.

Problem 2: the stack is small. Local variables live in a region of memory called the **stack**. It's typically 1 MB (Windows) to 8 MB (Linux): plenty for a few numbers, nowhere near enough for big data.

1.3 Overflowing the stack

Listing 1 `stack_overflow.cpp`

```cpp
#include <iostream>

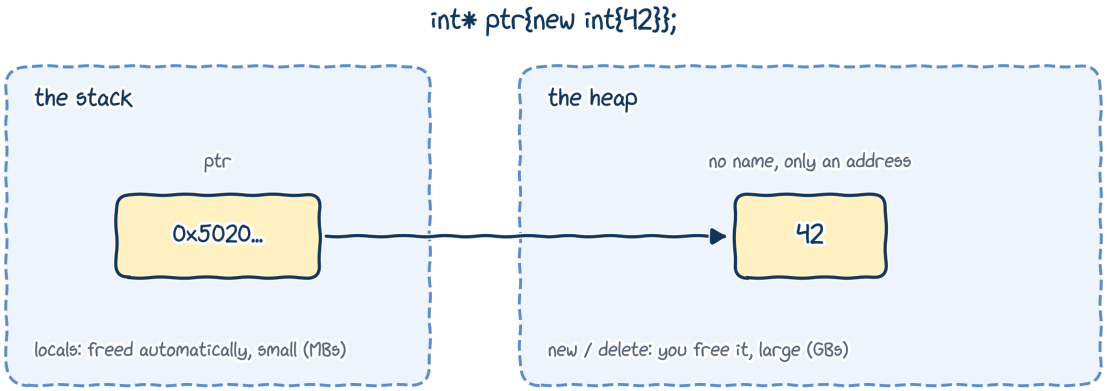
int main() {
    int numbers[10'000'000]; // 10 million ints (40 MB) on the stack
    numbers[0] = 1;
    std::cout << numbers[0] << '\n';
    return 0;
}
```

```
$ ./stack_overflow
Segmentation fault
```

`int numbers[10'000'000];` is a *C-style array* of 10 million `int`s (arrays: later in this chapter). At 40 MB, it doesn't fit on my 8 MB stack: a **stack overflow**, and the program crashes.

1.4 The heap

Dynamic memory allocation lets a running program ask the operating system for memory, whenever it likes and as much as it needs.



That memory comes from a much bigger region called the **heap** (or *free store*): gigabytes on a modern machine. Its objects have **no names**, only addresses, and that's what pointers are for!

2 `new` and `delete`

2.1 Allocating one object with `new`

Listing 2 `new_single.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{new int}; // an int on the heap
    *ptr = 7;          // use it through the pointer
    std::cout << *ptr << '\n';

    int* ptr2{new int{6}}; // allocate AND initialize
    std::cout << *ptr2 << '\n';

    delete ptr; // give the memory back
    delete ptr2;
    return 0;
}
```

```
7
6
```

- `new int` creates an `int` on the heap and returns its **address**, an `int*`. Keep it in a pointer, or it's lost.
- `new int{6}` creates it *and* initializes it, with braces as usual.

2.2 Giving memory back with `delete`

When you're done with a heap object, **you** must give its memory back:

```cpp
delete ptr;    // return the memory ptr points at
ptr = nullptr; // ptr now says "nothing allocated"
```

- Despite its name, `delete` doesn't delete the *pointer*. The pointer `ptr` is an ordinary local variable and lives on. `delete` destroys the **object it points at** and hands the memory back.
- Only `delete` what came from `new`. Deleting anything else is undefined behaviour.

> **!** Key idea
>
> Local variables are freed automatically. Heap objects are not: every `new` needs exactly one matching `delete`.

2.3 `delete` and `nullptr`

Listing 3 `delete_and_nullptr.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{new int{42}};
    std::cout << *ptr << '\n';

    delete ptr;    // the int is gone, but ptr still holds its address
    ptr = nullptr; // so say so: "nothing allocated"

    delete ptr; // deleting a null pointer is safe: it does nothing

    if (!ptr) {           // nothing allocated yet?
        ptr = new int{7}; // then allocate
    }
    std::cout << *ptr << '\n';
    delete ptr;
    return 0;
}
```

```
42
7
```

> **Best practice**
>
> After `delete`, set the pointer to `nullptr`, unless it's about to go out of scope anyway.

A null pointer is a handy way to say "nothing allocated yet". And `delete` on a null pointer does nothing, so there's no need to write `if (ptr) delete ptr;`.

2.4 Dangling pointers, again

Listing 4 `use_after_delete.cpp`

```cpp
#include <iostream>

int main() {
    int* ptr{new int{7}};
    delete ptr; // ptr is now dangling

    std::cout << *ptr << '\n'; // undefined behaviour: use after delete
    delete ptr;                // undefined behaviour: delete twice
    return 0;
}
```

```
$ ./use_after_delete
1573893628
free(): double free detected in tcache 2
Aborted
```

After `delete`, `ptr` still holds the old address: it's a **dangling pointer** (Chapter 08). Reading through it printed garbage; deleting it again crashed the program. Both are undefined behaviour.

2.5 Two pointers, one object

Dangling pointers are easy to create without noticing:

```cpp
int* owner{new int{5}};
int* viewer{owner}; // two pointers to the same heap int

delete owner;       // the int is gone...
owner = nullptr;    // ...and owner says so

// ...but viewer still holds the old address: it dangles!
```

> Best practice
>
> Avoid having several pointers to the same heap object. If you must, be clear about which one **owns** it: the owner is the only one that deletes it.

3 Memory leaks

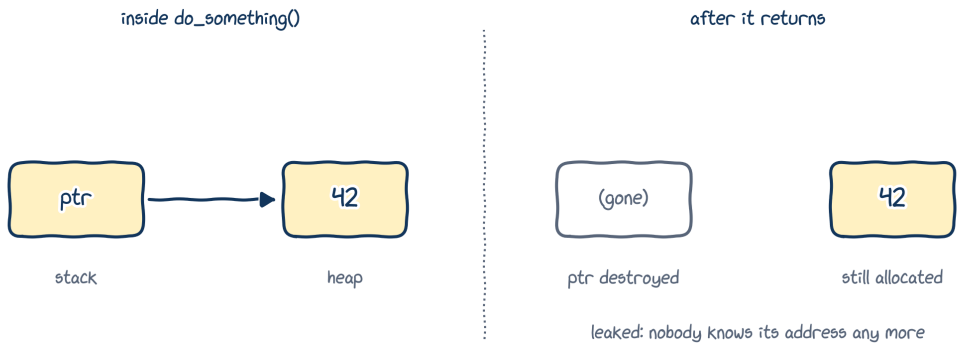
3.1 A memory leak

Listing 5 `memory_leak.cpp`

```cpp
void do_something() {
    int* ptr{new int{42}}; // allocated...
} // ptr is destroyed: the int leaks

int main() {
    do_something();
    return 0;
}
```

The pointer `ptr` is a local variable: it's destroyed at the `}`. The heap `int` is not. Nobody holds its address any more, so nobody can ever `delete` it.



3.2 Why leaks matter

- A leaked object stays allocated until the **program ends**. The operating system can't reuse that memory either: as far as it knows, your program is still using it.
- One leaked `int` is harmless. A leak inside a loop that runs a million times, in a simulation that runs for a week, eats all the memory and slows down the whole machine.
- Leaks make no noise: the program still "works". You need a tool to find them.

> **i** Coming from Python
>
> Python has a **garbage collector** that frees objects nobody refers to any more. C++ has none: if you `new` it, you `delete` it. (Or let something else do it for you: the end of this chapter.)

3.3 Catching leaks with AddressSanitizer

g++ can build your program with **AddressSanitizer**, a checker for memory bugs:

```
$ g++ -std=c++20 -g -fsanitize=address memory_leak.cpp
$ ./a.out
==6651==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 4 byte(s) in 1 object(s) allocated from:
```

```
    #1 in do_something() memory_leak.cpp:2
SUMMARY: AddressSanitizer: 4 byte(s) leaked in 1 allocation(s).
```

It even tells you which line allocated the leaked memory. (Output shortened: the real report also shows addresses and full paths.) It catches dangling pointers too: on `use_after_delete.cpp` it reports *heap-use-after-free* on line 7.

> Best practice
>
> While developing, compile with `-g -fsanitize=address` now and then. It's free bug-hunting. (It works on Linux and macOS; on Windows, MSVC has its own version.)

3.4 Three ways to leak

1. **The pointer goes out of scope** (the last slides).
2. **The pointer is reassigned:**

```
int value{5};
int* ptr{new int{}};
ptr = &value; // the heap int is lost
```

3. **Allocating twice:**

```
int* ptr{new int{}};
ptr = new int{}; // the first int is lost
```

The fix for 2 and 3: `delete` before you reassign.

```
int* ptr{new int{}};
delete ptr;      // give it back first
ptr = new int{}; // now this is fine
```

4 Dynamic arrays

4.1 Allocating an array with `new[]`

Listing 6 `dynamic_array.cpp`

```cpp
#include <iostream>

int main() {
    std::cout << "How many numbers? ";
    int length{};
    std::cin >> length; // known only when the program runs

    int* numbers{new int[length]{}}; // length ints, all set to 0

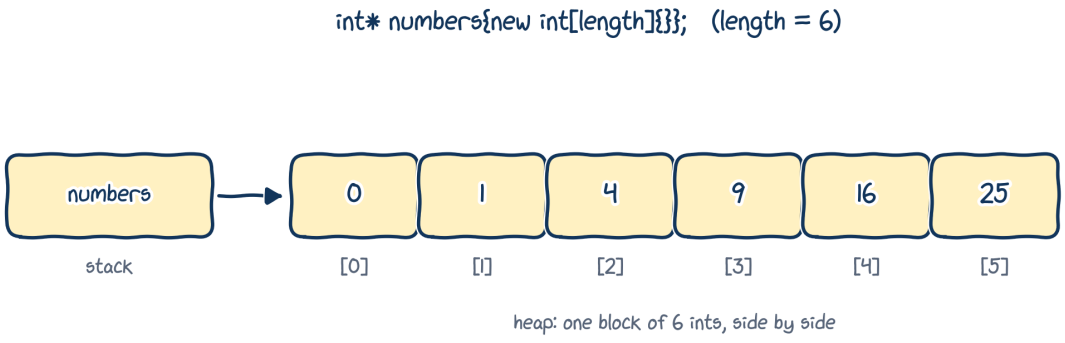
    for (int i{0}; i < length; ++i) {
        numbers[i] = i * i; // the i-th int
    }
    for (int i{0}; i < length; ++i) {
        std::cout << numbers[i] << ' ';
    }
    std::cout << '\n';

    delete[] numbers; // array new needs array delete
    return 0;
}
```

```
$ ./dynamic_array
How many numbers? 6
0 1 4 9 16 25
```

The length can be **any value known at run time**: exactly what automatic memory couldn't do.

4.2 What `new int[length]` gives you



- One block of `length` `int`s, side by side, and a pointer to the first one.
- `numbers[i]` is the `int` at position `i`, counting from 0, as with `std::string` in Chapter 04. There's no checking: `numbers[length]` is out of bounds and undefined behaviour.
- `new int[5]{}` sets them all to 0; `new int[5]{9, 7, 5, 3, 1}` gives them values.

4.3 `new[]` needs `delete[]`

Listing 7 `delete_mismatch.cpp`

```cpp
int main() {
    int* numbers{new int[5]{}};
    delete numbers; // wrong: this should be delete[]
    return 0;
}
```

```
warning: 'void operator delete(void*, long unsigned int)' called on
pointer returned from a mismatched allocation function [-Wmismatched-new-delete]
```

- Memory from `new[]` must be returned with `delete[]`, and memory from `new` with plain `delete`. Mixing them up is undefined behaviour.
- It compiles! Only `-Wall` warns, and AddressSanitizer reports *alloc-dealloc-mismatch (operator new [] vs operator delete)*.

4.4 `new` can fail

Listing 8 `new_can_fail.cpp`

```cpp
#include <iostream>
#include <new> // for std::nothrow

int main() {
    long long huge{1'000'000'000'000}; // a trillion ints: 4 TB!

    int* safe{new (std::nothrow) int[huge]}; // null if it fails
    if (!safe) {
        std::cout << "Could not allocate memory\n";
    }

    int* unsafe{new int[huge]}; // throws if it fails
    std::cout << "We never get here\n";

    delete[] safe;
    delete[] unsafe;
    return 0;
}
```

```
$ ./new_can_fail
Could not allocate memory
terminate called after throwing an instance of 'std::bad_alloc'
  what():  std::bad_alloc
Aborted
```

If the memory isn't available, `new` throws an **exception**, `std::bad_alloc` (exceptions are beyond this course), and the program stops. `new (std::nothrow)` returns `nullptr` instead, so you can check. For everyday sizes, failure is rare.

5 Let someone else `delete`

5.1 `std::vector`: a dynamic array that cleans up after itself

Listing 9 `vector_instead.cpp`

```cpp
#include <iostream>
#include <vector>

int main() {
    std::cout << "How many numbers? ";
    int length{};
    std::cin >> length;

    std::vector<int> numbers(length);

    for (int i{0}; i < length; ++i) {
        numbers[i] = i * i;
    }
    for (int i{0}; i < length; ++i) {
        std::cout << numbers[i] << ' ';
    }
    std::cout << '\n';
    return 0;
} // no delete[]: the vector frees its own memory here
```

```
$ ./vector_instead
How many numbers? 6
0 1 4 9 16 25
```

Note the **parentheses**: `numbers(length)` makes `length` zeros; `numbers{length}` would hold the single value `length`.

You've used `std::vector` in the labs. Inside, it does exactly what we did by hand: `new[]` to get memory, and `delete[]` when it's destroyed. You can't forget the `delete[]`, because you never write it.

5.2 Smart pointers: `std::unique_ptr`

Listing 10 `unique_ptr_example.cpp`

```cpp
#include <iostream>
#include <memory> // for std::unique_ptr and std::make_unique

int main() {
    std::unique_ptr<int> ptr{std::make_unique<int>(42)};
    std::cout << *ptr << '\n'; // use it just like a pointer
    return 0;
} // no delete: the int is freed automatically here
```

- A `std::unique_ptr` is a **smart pointer**: it owns one heap object and deletes it automatically when it goes out of scope. No leaks, no forgotten `delete`.
- `std::make_unique<int>(42)` does the `new` for you.
- How can an object clean up after itself? With a **destructor**, a special function that runs when an object is destroyed. We'll write our own in Chapter 12.

5.3 So why learn `new` and `delete` at all?

> Best practice
>
> In your own code, prefer `std::vector` for dynamic arrays and `std::unique_ptr` (with `std::make_unique`) for single heap objects. Write `new` and `delete` by hand only when you have a good reason.

But you still need to understand them:

- they're what `std::vector`, `std::string` and smart pointers do **under the hood**;
- you'll meet them in older code, in libraries, and when we write classes that manage memory themselves (Chapter 12);
- they make clear what "owning" an object means, an idea that runs through all of modern C++.

6 Wrapping up

6.1 Summary

- Local variables live on the small **stack** and are freed automatically. **Dynamic** objects live on the large **heap**, and *you* manage them.
- `new` creates an object on the heap and returns its address; `delete` gives the memory back. Every `new` needs exactly one `delete`.
- After `delete`, the pointer **dangles**: set it to `nullptr`. Deleting `nullptr` is safe.
- Losing the last pointer to a heap object is a **memory leak**. Find leaks with `-fsanitize=address`.
- `new int[n]` makes an array whose length is known only at run time; free it with `delete[]`, never plain `delete`.
- In practice, let `std::vector` and `std::unique_ptr` do the deleting for you.

6.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Dynamic memory allocation with `new` and `delete`
2. Dynamically allocating arrays
3. Introduction to `std::vector`
4. `std::unique_ptr` and smart pointers

Next up, Section 3: object-oriented programming, starting with Chapter 10.